Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт информационных технологий и кибербезопасности
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

дисциплины
«Программирование на Python»

Выполнил:
Сотников Родион Геннадьевич
2 курс, группа ИВТ-б-о-25-1
09.03.01 «Информатика и вычислительная техника», направленность (профиль) «Программное обеспечение робототехнических и интеллектуальных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института информационных технологий и кибербезопасности Воронкин Р.А.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Ссылка на репозиторий: https://github.com/rodions016-star/python-lab-03

Тема работы: Работа со строками в Python.

Цель работы: освоить создание и обработку строк, индексацию и срезы, поиск и замену подстрок, преобразование регистра, разделение и объединение текста; научиться выбирать строковые методы и проверять граничные случаи.

Рабочая среда: Windows 11; Python 3.13.14; редактор Zed 1.23.2. Для отладки создано виртуальное окружение .venv с пакетом debugpy, сами программы используют только стандартную библиотеку.

Варианты: вариант 17 в каждом из заданий 1, 2 и 3 (по одному варианту из каждой обязательной группы).

Конспект теоретического материала

Строка str — неизменяемая последовательность символов Unicode; отдельного типа «символ» нет, результат индексации — строка длины 1. Литералы записываются в апострофах или кавычках, управляющие символы задаются последовательностями \n, \t, \\; префикс r отключает экранирование, тройные кавычки допускают несколько строк. Операторы: + склеивает строки, * повторяет, in и not in проверяют наличие подстроки. Функция len() возвращает число символов, ord() и chr() переводят символ в код и обратно, str() даёт строковое представление объекта. Функция input() возвращает строку без перевода строки, но с пробелами по краям, поэтому strip() к вводу автоматически не применяют; для просмотра невидимых символов используют repr().

Символы нумеруются с нуля, отрицательные индексы считаются с конца (−1 — последний). Индекс вне строки вызывает IndexError. Срез s[start:stop:step] берёт символы от start включительно до stop не включая и не выходит за границы ошибкой; s[::-1] разворачивает строку, нулевой шаг

запрещён. Строки неизменяемы, поэтому методы не меняют исходную строку, а возвращают новую, которую нужно сохранить: result = text.replace(" ", "_").

Основные методы: find() и count() ищут подстроку (find() возвращает −1, если подстроки нет, и 0 для совпадения в начале), replace() заменяет подстроки, lower() и upper() меняют регистр, isalpha(), isdecimal(), isspace() проверяют состав строки, split() делит текст на слова, join() собирает строку из списка. Для вывода значений применяют f-строки: перед открывающей кавычкой ставится f, а выражения пишутся в фигурных скобках. В индивидуальных заданиях регулярные выражения не используются.

Ход выполнения работы

1. Подготовка проекта. Создан репозиторий python-lab-03 с файлом README.md и виртуальное окружение .venv (python -m venv .venv). В корне создан файл .gitignore. Во всех программах основной сценарий находится под if __name__ == "__main__":, импорты стоят перед ним.

Листинг 1 — Файл .gitignore

```
.venv/
__pycache__/
*.py[cod]
```

В таблицах проверок значения строк показаны в записи Python (в кавычках), чтобы были видны пробелы по краям и табуляция \t; сами кавычки в программу не вводятся. Если входных значений несколько, они перечислены в порядке ввода, каждое вводится отдельным вызовом input(). Ожидаемый результат записан до запуска программ.

2. Пример 1 — замена пробелов (replace_spaces.py). Вводится строка, каждый обычный пробел заменяется символом подчёркивания; табуляция и другие символы не меняются, пустая строка допустима.

Листинг 2 — Программа replace_spaces.py

```
if __name__ == "__main__":
    text = input("Введите предложение: ")
    result = text.replace(" ", "_")
    print(f"Предложение после замены: {result}")
```

Метод replace() возвращает новую строку, поэтому результат сохраняется в переменной result; значение text остаётся прежним. Пробелы по краям не удаляются, а тоже заменяются. Префикс f перед строкой вывода нужен, чтобы вместо текста {result} напечаталось значение переменной.

Таблица 1 — Проверка программы replace_spaces.py

Входные данные	Ожидаемый результат	Фактический результат	Вывод
'мама мыла раму'	'мама_мыла_раму'	'мама_мыла_раму'	Совпадает
' а б '	'_а_б_'	'_а_б_'	Совпадает
''	''	''	Совпадает
'а\tб'	'а\tб'	'а\tб'	Совпадает
'слово'	'слово'	'слово'	Совпадает
'а б'	'а__б'	'а__б'	Совпадает

3. Пример 2 — удаление середины слова (remove_middle.py). Из непустого слова удаляется средняя буква при нечётной длине или две средние буквы при чётной. Пустой ввод запрещён и проверяется.

Листинг 3 — Программа remove_middle.py

```
import sys


if __name__ == "__main__":
    word = input("Введите слово: ")
    if not word:
        print("Ошибка: слово не должно быть пустым", file=sys.stderr)
        sys.exit(1)

    middle = len(word) // 2
    if len(word) % 2 == 1:
        result = word[:middle] + word[middle + 1:]
    else:
        result = word[:middle - 1] + word[middle + 1:]

    print(f"Результат: {result}")
```

Пустая строка в условии if not word: считается ложной, поэтому сообщение выводится в поток ошибок, а программа завершается командой sys.exit(1). При нечётной длине middle — индекс единственной средней буквы: левый срез word[:middle] заканчивается перед ней, правый word[middle + 1:] начинается после неё. При чётной длине средние буквы имеют индексы middle − 1 и middle, поэтому левый срез word[:middle - 1] заканчивается на одну букву раньше. Части соединяются конкатенацией. Для слов длины 1 и 2 результат пустой — это корректный ответ, в отличие от пустого ввода.

Таблица 2 — Проверка программы remove_middle.py

Входные данные	Ожидаемый результат	Фактический результат	Вывод
'кот'	'кт'	'кт'	Совпадает
'мама'	'ма'	'ма'	Совпадает
'я'	''	''	Совпадает
'мы'	''	''	Совпадает
''	Ошибка: слово не должно быть пустым (код 1)	Ошибка: слово не должно быть пустым (код 1)	Совпадает
'слово'	'слво'	'слво'	Совпадает
'абвгде'	'абде'	'абде'	Совпадает

4. Пример 3 — равномерное распределение пробелов (justify_text.py). Вводятся строка из двух или более слов и требуемая длина target_length. Слова и их порядок сохраняются, пробелы между ними перераспределяются так, чтобы длины промежутков отличались не более чем на 1, а более длинные промежутки стояли левее. Пробелов по краям в результате нет; длина результата должна быть не меньше исходной и не больше 10000.

Листинг 4 — Программа justify_text.py

```
import sys


if __name__ == "__main__":
    text = input("Введите предложение: ")
    target_length = int(input("Введите требуемую длину: "))

    if text != text.strip(" ") or any(
        char.isspace() and char != " " for char in text
    ):
        print("Ошибка: допустимы только пробелы между словами", file=sys.stderr)
        sys.exit(1)

    words = text.split()
    if len(words) < 2:
        print("Ошибка: требуется не менее двух слов", file=sys.stderr)
        sys.exit(1)

    if not len(text) <= target_length <= 10000:
        print("Ошибка: требуется len(text) <= длина <= 10000", file=sys.stderr)
        sys.exit(1)

    letters_length = 0
    for word in words:
        letters_length += len(word)

    spaces_total = target_length - letters_length
    gaps = len(words) - 1
    base_width = spaces_total // gaps
    extra_gaps = spaces_total % gaps

    parts = []
    for index, word in enumerate(words):
        parts.append(word)
        if index < gaps:
            width = base_width
```

```
            if index < extra_gaps:
                width += 1
            parts.append(" " * width)

    result = "".join(parts)
    print(f"Результат: {result}")
    print(f"Длина: {len(result)}")
```

Сначала проверяется ввод: пробелы по краям (text != text.strip(" ")) и любые пробельные символы, кроме обычного пробела (в выражении any(...) каждый символ проверяется по очереди). Затем split() без аргументов создаёт список настоящих слов без пустых элементов, даже если между словами несколько пробелов. Далее проверяются число слов и требуемая длина. Цикл for суммирует длины слов в letters_length, на пробелы остаётся spaces_total = target_length − letters_length. Между n словами ровно n − 1 = gaps промежутков; целочисленное деление даёт базовую ширину base_width, а остаток extra_gaps — число промежутков, в которые надо добавить ещё один пробел. Функция enumerate() даёт индекс и слово: после каждого слова, кроме последнего, в список parts добавляется промежуток нужной ширины. Метод join() с пустым разделителем собирает результат, так как все пробелы уже находятся в списке.

Для строки «мама мыла раму» и длины 17 длина слов равна 12, на пробелы остаётся 5, промежутков два: base_width = 2, extra_gaps = 1. Поэтому первый промежуток имеет ширину 3, второй — 2. Трассировка цикла приведена в таблице.

Таблица 3 — Трассировка justify_text.py для «мама мыла раму» и длины 17

Итерация	**index**	**word**	**Ширина промежутка**	**parts после итерации**	**Причина перехода к следующей**
1	0	мама	3	`['мама', '   ', 'мыла']`	0 < 1 истинно: ширина base_width + 1 = 3
2	1	мыла	2	`['мама', '   ', 'мыла', '  ', 'раму']`	1 < 1 ложно: ширина base_width = 2
3	2	раму	—	`['мама', '   ', 'мыла', '  ', 'раму']`	2 < 2 ложно: после последнего слова пробел не добавляется

Длина результата правильная, потому что сумма длин слов 12 и ширин промежутков 3 + 2 = 5 равна 17. В общем случае base_width · gaps + extra_gaps

= spaces_total, то есть ровно extra_gaps промежутков получают ширину base_width + 1, остальные — base_width, а все пробелы вместе занимают spaces_total позиций.

Таблица 4 — Проверка программы justify_text.py

Входные данные	Ожидаемый результат	Фактический результат	Вывод
'два слова'; 9	'два слова', длина 9	'два слова', длина 9	Совпадает
'мама мыла раму'; 17	'мама мыла раму', длина 17	'мама мыла раму', длина 17	Совпадает
'мама мыла раму'; 14	'мама мыла раму', длина 14	'мама мыла раму', длина 14	Совпадает
'мама мыла раму'; 16	'мама мыла раму', длина 16	'мама мыла раму', длина 16	Совпадает
'а б'; 6	'а б', длина 6	'а б', длина 6	Совпадает
'а б в'; 6	'а б в', длина 6	'а б в', длина 6	Совпадает
'а б'; 5	'а б', длина 5	'а б', длина 5	Совпадает
'два слова'; 8	Ошибка: требуется len(text) <= длина <= 10000 (код 1)	Ошибка: требуется len(text) <= длина <= 10000 (код 1)	Совпадает
'слово'; 10	Ошибка: требуется не менее двух слов (код 1)	Ошибка: требуется не менее двух слов (код 1)	Совпадает
' два слова'; 12	Ошибка: допустимы только пробелы между словами (код 1)	Ошибка: допустимы только пробелы между словами (код 1)	Совпадает
'два слова '; 10	Ошибка: допустимы только пробелы между словами (код 1)	Ошибка: допустимы только пробелы между словами (код 1)	Совпадает
'a\tб'; 3	Ошибка: допустимы только пробелы между словами (код 1)	Ошибка: допустимы только пробелы между словами (код 1)	Совпадает
''; 0	Ошибка: требуется не менее двух слов (код 1)	Ошибка: требуется не менее двух слов (код 1)	Совпадает

Строка «мама мыла раму» при длине 14 остаётся прежней (пробелов столько же), при длине 16 каждый промежуток равен 2. Случаи с ошибками завершаются кодом 1 и сообщением в потоке ошибок.

5. Наблюдение в отладчике. Программа justify_text.py запущена в отладчике Zed (адаптер debugpy, интерпретатор из .venv), введены строка «мама мыла раму» и длина 17. Точки останова установлены на строке 35 (if index < gaps: — в начале тела цикла) и на строке 39 (parts.append(" " * width) — добавление промежутка). На первой остановке на строке 35: index = 0, word = 'мама', parts = ['мама'], spaces_total = 5, gaps = 2, base_width = 2, extra_gaps = 1. Следующая остановка — строка 39: width = 3, так как 0 < extra_gaps; после

шага (Step Over) в parts добавлен промежуток из трёх пробелов. Вторая итерация: на строке 35 index = 1, word = 'мыла'; на строке 39 width = 2, так как 1 < 1 ложно. Третья итерация: index = 2, word = 'раму', условие 2 < gaps ложно, строка 39 пропущена, цикл заканчивается, и строка 41 даёт result = 'мама мыла раму' длиной 17.

6. Индивидуальное задание 1 (вариант 17). Формулировка: дано предложение; определить число вхождений в него некоторого символа. Предусловия: предложение может быть пустым, символ вводится отдельным вызовом input() и имеет длину 1; регистр учитывается, поэтому «м» и «М» различны, как и кириллическая «а» и латинская «a».

Листинг 5 — Программа individual_01.py

```
if __name__ == "__main__":
    text = input("Введите предложение: ")
    symbol = input("Введите символ: ")

    count = text.count(symbol)
    print(f"Число вхождений: {count}")
```

Метод count() считает вхождения подстроки без перекрытия; для подстроки длины 1 перекрытий не бывает, поэтому метод подходит и не требует цикла. Если символа в предложении нет или предложение пустое, count() возвращает 0, а по общему правилу для количества именно 0 и нужно выводить, так что отдельной ветви «не найдено» нет. Вызов strip() к вводу не применяется: иначе символ-пробел нельзя было бы подсчитать. Порядковые номера и индексы в этой программе не используются.

Таблица 5 — Проверка программы individual_01.py

Входные данные	Ожидаемый результат	Фактический результат	Вывод
'мама мыла раму'; 'м'	4	4	Совпадает
'Мама'; 'м'	1	1	Совпадает
''; 'а'	0	0	Совпадает
'привет'; 'я'	0	0	Совпадает
'а б а'; ' '	2	2	Совпадает
'aaa'; 'a'	3	3	Совпадает
'a-b-c'; '-'	2	2	Совпадает
'aa'; 'a'	1	1	Совпадает
'аа'; 'а'	1	1	Совпадает

7. Индивидуальное задание 2 (вариант 17). Формулировка: дано предложение; заменить в нём все вхождения буквосочетания «ах» на «ух». Предусловия: предложение может быть пустым; заменяется только сочетание из строчных кириллических букв, поэтому «Ах» и латинское «ах» не меняются.

Листинг 6 — Программа individual_02.py

```
if __name__ == "__main__":
    text = input("Введите предложение: ")

    result = text.replace("ах", "ух")

    if result:
        print(f"Результат: {result}")
    else:
        print(f"Результат: пустая строка {result!r}")
```

Метод replace() заменяет все непересекающиеся вхождения за один проход слева направо по исходной строке, а новая строка «ух» не содержит «ах», поэтому повторных замен нет. Возвращённое значение сохраняется в result, так как исходная строка не меняется. Пустой результат (получается только из пустого ввода) выводится с подписью и repr(), чтобы отличить его от отсутствия вывода. Отсутствие совпадений не ошибка: строка возвращается без изменений. Индексы и номера не используются.

Таблица 6 — Проверка программы individual_02.py

Входные данные	Ожидаемый результат	Фактический результат	Вывод
'ах, как страшно'	'ух, как страшно'	'ух, как страшно'	Совпадает
'Ахах'	'Ахух'	'Ахух'	Совпадает
'ахах'	'ухух'	'ухух'	Совпадает
'аах'	'аух'	'аух'	Совпадает
'привет'	'привет'	'привет'	Совпадает
''	''	''	Совпадает
'ах'	'ах'	'ах'	Совпадает
'ах мах'	'ух мух'	'ух мух'	Совпадает

8. Индивидуальное задание 3 (вариант 17). Формулировка: дано слово; переставить его s-ю букву на место k-й (s < k); при этом (s + 1)-ю, (s + 2)-ю, …, k-ю буквы сдвинуть влево на одну позицию. Предусловия: слово непустое, числа s и k целые, 1 ≤ s < k ≤ длина слова. Методичка не требует проверять

нарушения предусловий; в программе проверка всё же оставлена: при нарушении выводится сообщение в поток ошибок и код 1.

Листинг 7 — Программа individual_03.py

```
import sys


if __name__ == "__main__":
    word = input("Введите слово: ")
    s = int(input("Введите номер s: "))
    k = int(input("Введите номер k: "))

    if not 1 <= s < k <= len(word):
        print("Ошибка: требуется 1 <= s < k <= длина слова", file=sys.stderr)
        sys.exit(1)

    result = word[:s - 1] + word[s:k] + word[s - 1] + word[k:]
    print(f"Результат: {result}")
```

Номера букв считаются с 1, а индексы Python — с 0, поэтому s-я буква — это word[s - 1]. Буквы с (s + 1)-й по k-ю имеют индексы от s до k − 1, то есть образуют срез word[s:k]; всё после k-й буквы — срез word[k:], всё до s-й — word[:s - 1]. Результат склеивается из четырёх частей: начало, сдвинутые влево буквы, перенесённая буква, конец. Например, для «abcdef», s = 2, k = 5: «a» + «cde» + «b» + «f» = «acdebf». Длина результата равна длине слова: (s − 1) + (k − s) + 1 + (длина − k). Срезы не выходят за границы ошибкой, а одиночный индекс word[s - 1] выходит, поэтому проверка границ нужна. Отсутствующего результата в задаче нет. Достаточно срезов и конкатенации, циклы и методы не нужны.

Таблица 7 — Проверка программы individual_03.py

Входные данные	Ожидаемый результат	Фактический результат	Вывод
'abcdef'; 2; 5	'acdebf'	'acdebf'	Совпадает
'abcdef'; 1; 2	'bacdef'	'bacdef'	Совпадает
'abcdef'; 5; 6	'abcdfe'	'abcdfe'	Совпадает
'abcdef'; 1; 6	'bcdefa'	'bcdefa'	Совпадает
'процессор'; 3; 5	'прцеоссор'	'прцеоссор'	Совпадает
'аб'; 1; 2	'ба'	'ба'	Совпадает
'aabba'; 1; 3	'ababa'	'ababa'	Совпадает
'abc'; 3; 2	Ошибка: требуется 1 <= s < k <= длина слова (код 1)	Ошибка: требуется 1 <= s < k <= длина слова (код 1)	Совпадает
'abc'; 1; 4	Ошибка: требуется 1 <= s < k <= длина слова (код 1)	Ошибка: требуется 1 <= s < k <= длина слова (код 1)	Совпадает
'abc'; 2; 2	Ошибка: требуется 1 <= s < k <= длина слова (код 1)	Ошибка: требуется 1 <= s < k <= длина слова (код 1)	Совпадает

Входные данные	Ожидаемый результат	Фактический результат	Вывод
`''; 1; 2`	`Ошибка: требуется 1 <= s < k <= длина слова (код 1)`	`Ошибка: требуется 1 <= s < k <= длина слова (код 1)`	Совпадает

Ответы на вопросы для защиты работы

1) Что представляет собой строка str в Python? Чем она отличается от набора байтов?

str — неизменяемая последовательность символов Unicode (кодовых точек), предназначенная для хранения текста. Набор байтов (bytes) — последовательность чисел от 0 до 255; текст превращается в байты кодировкой, например UTF-8, причём русская буква в UTF-8 занимает два байта.

2) Чем отличаются обычные, сырые и многострочные литералы?

В обычном литерале обратный слеш начинает управляющую последовательность (\n — перевод строки). В сыром литерале с префиксом r слеши сохраняются как есть, поэтому он удобен, например, для путей Windows; он не может заканчиваться нечётным числом слешей. Многострочный литерал в тройных кавычках может занимать несколько строк и содержать одиночные кавычки.

3) Какие операции можно выполнять над строками с помощью +, *, in и not in?

+ склеивает две строки, * повторяет строку целое число раз (при нуле или отрицательном числе получается пустая строка), in возвращает True, если подстрока входит в строку, not in — обратное значение.

4) Как работают положительные и отрицательные индексы? Что происходит при выходе за границы?

Положительные индексы идут с нуля слева, отрицательные — справа: −1 — последний символ. Индекс вне диапазона, например s[len(s)], вызывает IndexError.

5) Как устроен срез start:stop:step? Чем границы среза отличаются от одиночного индекса?

Срез берёт символы от start включительно до stop не включая с шагом step. Одиночный индекс возвращает один символ, а при выходе за границы вызывает ошибку; срез ошибки не вызывает и при необходимости возвращает пустую строку.

6) Что означает неизменяемость строки? Почему присваивание s = s.replace(...) допустимо?

Символы существующей строки менять нельзя (s[0] = "x" вызовет TypeError), а методы возвращают новую строку. Присваивание лишь связывает имя s с новой строкой, старая при этом не изменяется.

7) Чем отличаются capitalize() и title()? Почему они не заменяют лингвистическую обработку текста?

capitalize() делает заглавной первую букву строки, остальные строчными, title() — первую букву каждого слова. Методы не знают языка: "they're".title() даёт "They'Re", имена, аббревиатуры и исключения обрабатываются неверно.

8) Когда для поиска достаточно in, а когда нужен find()?

Оператора in достаточно, когда нужен ответ «есть или нет». Метод find() нужен, когда требуется позиция вхождения; он возвращает −1, если подстроки нет.

9) Чем различаются find(), index(), rfind() и rindex()?

find() и index() ищут первое вхождение слева, rfind() и rindex() — последнее. При отсутствии подстроки find() и rfind() возвращают −1, а index() и rindex() вызывают ValueError.

10) Что измеряет len() для строки? Всегда ли это количество видимых знаков?

len() считает кодовые точки Unicode. Видимый знак может состоять из нескольких кодовых точек: «е» с отдельным знаком ударения даёт длину 2, хотя виден один знак.

11) Как count() обрабатывает перекрывающиеся вхождения и пустую подстроку?

count() считает только непересекающиеся вхождения: "aaa".count("aa") равно 1. Для пустой подстроки результат равен длине строки плюс 1: "abc".count("") даёт 4.

12) Как устроена f-строка? Как вывести фигурные скобки буквально?

Перед открывающей кавычкой ставится f, а выражения в фигурных скобках вычисляются и подставляются в текст. Буквальные скобки удваиваются: f"{{}}" печатает {}.

13) Как ограничить поиск параметрами start и end?

Методы find(), count() и другие принимают необязательные границы: s.find("a", 2, 10) ищет только в части строки от индекса 2 до индекса 10 (не включая его); возвращаемый индекс считается от начала всей строки.

14) Как пользоваться format() и спецификациями точности и ширины поля?

В фигурных скобках после двоеточия задаётся формат: "{:>8.3f}".format(3.14159) даёт «3.142», дополненное слева пробелами до ширины 8 (выравнивание вправо, три знака после точки); "<" — влево, "^" — по центру. Те же спецификации действуют в f-строках: f"{x:8.3f}".

15) Чем отличаются isdecimal(), isdigit() и isnumeric()? Почему isdigit() не проверяет произвольную запись числа?

isdecimal() истинно для десятичных цифр; isdigit() принимает ещё, например, надстрочную «²»; isnumeric() — также дроби вроде «½». Все три

проверяют только символы-цифры: строки "-5" и "3.14" дают False из-за знака и точки, поэтому запись числа этими методами не проверяют.

16) Чем отличается split() от split(' '), в том числе для пустой строки?

split() без аргумента делит по любым пробельным символам и не создаёт пустых элементов; "".split() даёт []. split(" ") делит по каждому одиночному пробелу, поэтому строка из «а», двух пробелов и «б» даёт ["а", "", "б"], а "".split(" ") — [""].

17) Что проверяет islower()? Что получится для строки из одних цифр?

islower() истинно, если в строке есть хотя бы одна буква и все буквы строчные. Для "123" результат False, так как букв нет.

18) Как безопасно проверить регистр первого символа, если строка может быть пустой?

Выражением s[:1].islower() или s and s[0].islower(). Срез s[:1] для пустой строки даёт пустую строку, а индекс s[0] вызвал бы IndexError.

19) Почему нельзя непосредственно сложить строку и целое число? Какие есть способы получить текстовый результат?

Оператор + не преобразует типы: "5" + 3 вызывает TypeError. Число превращают в строку через str(3) или f-строку f"5{3}", либо строку в число через int("5") + 3.

20) Как развернуть строку с помощью среза? Почему шаг среза не может быть нулевым?

Срезом s[::-1]. Шаг 0 означал бы, что движения по строке нет, поэтому Python вызывает ValueError.

21) Почему join() вызывается у разделителя? Какие элементы он принимает?

Метод принадлежит строке-разделителю: ", ".join(items) вставляет её между элементами. Принимается итерируемый объект, все элементы которого — строки; число среди элементов вызовет TypeError.

22) Для чего нужен casefold() и чем он отличается от lower()?

casefold() предназначен для сравнения без учёта регистра и «сворачивает» больше символов: "ß".casefold() даёт "ss", а lower() оставляет "ß".

23) Как преобразовать первый и последний символы, учитывая пустую строку и строку длины 1?

Нужны три ветви: пустая строка остаётся без изменений; при длине 1 символ преобразуется один раз; иначе s[0].upper() + s[1:-1] + s[-1].upper(). Без этого в строке длины 1 единственный символ обработался бы дважды, а для пустой строки индекс вызвал бы ошибку.

24) Что проверяет isupper()? Как учитываются цифры и пунктуация?

isupper() истинно, если есть хотя бы одна буква и все буквы заглавные. Цифры и пунктуация игнорируются: "A1!" даёт True, а "123" — False.

25) Когда полезен splitlines()? Что делает аргумент keepends?

splitlines() делит текст по границам строк (\n, \r\n и другим) и не создаёт пустого элемента после последнего перевода. С keepends=True символы конца строки остаются в элементах.

26) Как работает replace() с ограничением числа замен?

Третий аргумент задаёт максимум замен слева направо: "a-b-c".replace("-", "+", 1) даёт "a+b-c".

27) Когда применяются startswith(), endswith(), removeprefix() и removesuffix()?

startswith() и endswith() проверяют начало и конец строки. removeprefix() и removesuffix() (Python 3.9+) удаляют заданный префикс или суффикс, только если он есть, и один раз.

28) Чем отличаются обычный пробел и пробельные символы? Что проверяет isspace()?

Обычный пробел — один символ " ". Пробельные символы — все, для которых isspace() истинно: пробел, табуляция, перевод строки, неразрывный пробел и другие. isspace() истинно для непустой строки, состоящей только из таких символов.

29) Почему strip(chars) не является удалением заданного префикса или суффикса?

strip(chars) снимает с краёв любые символы из набора chars, а не подстроку целиком: "xabcx".strip("xa") даёт "bc". Для удаления префикса или суффикса служат removeprefix() и removesuffix().

30) Как expandtabs() определяет количество пробелов для каждой табуляции?

Табуляция заменяется пробелами до ближайшей позиции, кратной размеру табуляции (по умолчанию 8), считая от начала строки: "a\tb".expandtabs(4) даёт «a», три пробела и «b».

31) Что возвращает partition(), если разделитель найден и если не найден?

Кортеж из трёх строк: часть до первого разделителя, сам разделитель и часть после. Если разделителя нет, результат (s, "", ""): "abc".partition("x") даёт ("abc", "", "").

32) Как выбрать тесты для строкового алгоритма и обнаружить ошибку на первой или последней позиции?

Взять пустую строку, один символ и минимальную допустимую длину, случаи без совпадений, совпадение в начале и в конце, несколько совпадений,

разные регистры, пробелы по краям, соседние повторы. Ожидаемый результат записывается до запуска. Ошибки «на единицу» в срезах проявляются именно при совпадении на первой и последней позициях.

Вывод

В ходе лабораторной работы освоены создание и обработка строк: индексация, срезы, методы count() и replace(), разделение и объединение текста. Разобраны три примера и выполнены три индивидуальные программы варианта 17: подсчёт вхождений символа, замена буквосочетания и перестановка буквы слова срезами. Для каждой программы составлено не менее пяти тестов с границами и пустыми строками; во всех 54 проверках фактический результат совпал с заранее записанным ожидаемым. Материалы размещены в репозитории python-lab-03.